FC Fluxo de Caixa

DOCUMENTO 05

Requisitos não funcionais

SLI, SLO, error budget, escala, resiliência e plano de carga

PROJETO

Fluxo de Caixa — Desafio de Arquiteto de Soluções

AUTOR

Bruno Aquino

DATA

11/11/2026

REPOSITÓRIO

github.com/brunoaquino/fluxo-caixa

Java 25 · Spring Boot 4 · AWS ECS Fargate · Terraform · C4 · DDD · ADRs

Sumário

1. SLI, SLO e error budget 3

2. Escalabilidade 4

3. Resiliência 4

4. Desempenho 5

5. Plano de teste de carga (k6) 6

6. Matriz de rastreabilidade 6

FONTE: DOCUMENTOS-BRUTOS/05-REQUISITOS-NAO-FUNCIONAIS.MD

05 — Requisitos não funcionais

Este documento operacionaliza os RNF do contexto e as metas do blueprint. Número sem teste é intenção; a coluna de evidência diz onde a intenção vira verificação.

1. SLI, SLO e error budget

Janela: 30 dias corridos (43.200 minutos). O SLI de disponibilidade e de latência do processo continua medido no serviço, dentro de `us-east-1`, e no próprio processo no local. O API Gateway acrescenta cerca de 10–30 ms nesse caminho (ADR 0031). Esse acréscimo não entra no SLO do processo: entra no tempo que o comerciante percebe, junto com o RTT entre o edge de São Paulo e a Virgínia (cerca de 110–130 ms numa conexão já quente, ADR 0021). Bloqueio do WAF e 401 do authorizer são séries à parte, não 5xx do serviço. Se um contrato futuro medir o SLO na borda, o orçamento de latência do processo encolhe esses 10–30 ms; hoje os dois relógios ficam separados de propósito.

Serviço	SLI	SLO	Error budget
Lançamentos	Proporção de minutos em que o POST e o GET respondem sem 5xx e sem timeout, com o consolidado saudável ou não	99,9%	43,2 min/mês
Consolidado	O mesmo para o GET de saldo e de período	99,5%	216 min/mês
Lançamentos	Latência do POST	p95 < 200 ms, p99 < 500 ms	estoura se o percentil passar por 1 h contínua no pico
Consolidado	Latência do GET diário	p95 < 100 ms com hit de cache; p99 < 300 ms	idem
Consolidado	Taxa de erro durante pico ≥ 50 req/s	≤ 5% (teto do enunciado); ≤ 1% (meta interna)	2,5 erros/s no teto; 0,5 erro/s na meta
Consolidação	`aplicadoEm − occurredAt`	p95 < 5 s em regime estável	não se aplica ao catch-up depois de indisponibilidade longa
Dados	RPO / RTO intra-região	≤ 5 min / ≤ 1 h	PITR e game day

Política de queima:

- Queima rápida: 2% do budget mensal em 1 hora (lançamentos ≈ 0,9 min de erro naquela hora; consolidado ≈ 4,3 min). Alarme page.
- Queima lenta: 5% do budget em 6 horas. Alarme ticket.
- Budget esgotado no mês: congela deploy que não seja correção de confiabilidade. O congelamento é regra de time, não de ferramenta; o ADR 0020 deixa o rollback técnico independente disso.

Latência e disponibilidade são SLOs separados. Um serviço rápido que fica fora estoura budget de disponibilidade. Um serviço no ar com p95 de 800 ms estoura o de latência e ainda pode estar “disponível”.

2. Escalabilidade

Escala horizontal primeiro.

Alavanca	Como está hoje	Limite em que muda o desenho
Tasks ECS	Prod: mínimo 2, máximo 10 nos dois serviços. Lançamentos 0,5 vCPU / 1 GiB. Consolidado 1 vCPU / 2 GiB. CPU, request do ALB e, no consolidado, backlog da fila	Acima do teto, rever pool JDBC (10 no lançamentos, 20 no consolidado, por task) contra `max_connections` do `db.t4g.medium`
Cache	Versão por comerciante, TTL 30 s, Redis local e Valkey na AWS	Hit ratio < 80% no pico: ou o padrão de acesso não é o da premissa P6, ou a versão está sendo incrementada à toa
Banco	`db.t4g.medium` Multi-AZ em prod, gp3 50 GB, 3.000 IOPS inclusos. Réplica desligada até o gatilho	Leitura do `SUM` : réplica, não instância maior. Ver 12
Fila	Standard, competing consumers = número de tasks	Se um comerciante sozinho gerar mais do que uma task drena, particionar por `comercianteId` (fila FIFO com message group, ou várias filas). Não é o caso a 20 evt/s
Particionamento de dado	Não há. Um banco cabe	Shard por `comerciante_id` só se um único Postgres não guardar o histórico. Com uma linha por lançamento e dezenas de milhões de linhas, ainda é cedo

Não há scale-to-zero em produção. O mínimo de duas tasks é o que torna o N−1 verdadeiro no pico de 50 req/s (ρ sobe de 17% para 33% com uma task, ainda abaixo do joelho). Scale-from-zero economizaria a task ociosa e gastaria o SLO no primeiro minuto do fechamento.

Auto scaling reage em minutos. Por isso o desired count do fechamento conhecido (dia de pico de calendário) pode ser elevado por schedule antes do evento, em vez de esperar a CPU. Isso é ajuste de operação, não de código.

3. Resiliência

Mecanismo	Onde	Efeito
Duas AZs, tasks espalhadas	ECS	Perda de uma zona não zera o serviço
RDS Multi-AZ	Cada banco	Promoção do standby sem mudar a string de conexão além do failover do endpoint
ElastiCache com réplica	Consolidado	Promoção do nó. Se o cluster inteiro some, o GET cai no Postgres

Mecanismo	Onde	Efeito
Fila com 14 dias	SQS	Consolidado pode ficar fora uma semana sem perder evento
Outbox	Lançamentos	SNS fora não desfaz o 201
Backoff do relay	`proxima_tentativa_em` até 60 s, mais Resilience4j no cliente SNS: retry com jitter, time limiter e circuit breaker (ADR 0023)	O POST não chama o SNS. Circuito aberto adia o lote; o evento fica na outbox
Timeout curto	Redis e chamadas AWS do relay	Thread virtual não fica presa; o POST não chama esses clientes
DLQ após 5 tentativas	SQS	Poison message sai da frente
Visibility 60 s	SQS	Processamento normal é de dezenas de milissegundos; 60 s cobre um GC ruim sem devolver a mensagem cedo demais
Bulkhead de pool	Hikari máximo 10	Stampede não abre 200 conexões
Degradação	Redis fora ou circuito aberto	Leitura segue no Postgres, mais lenta. Readiness continua verde (ADR 0020, ADR 0023)
Deploy circuit breaker	ECS	Task nova doente não substitui as saudáveis até o fim
Backoff do cliente do caixa	Fora do nosso processo	429/503 com `Retry-After` quando viermos a introduzir limite de aplicação; hoje o limite grosso é o WAF

O que não está coberto: perda da região `us-east-1`. O RTO de 1 h não é multi-região. Dizer o contrário no slide de venda seria falso; a evolução está no documento 10.

Game day mínimo, uma vez por trimestre, no dev AWS ou com falha injetada no local:

1. Parar o `consolidado-service` e registrar lançamento (espera-se 201).
2. Encher a fila, religar, medir tempo de catch-up.
3. Failover do RDS (reboot with failover) durante POST.
4. Redis inacessível durante GET (espera-se 200 mais lento, não 500 em massa).
5. Mensagem inválida na fila (espera-se DLQ em 5 recebimentos, fila principal seguindo).

O script `make resiliencia` cobre o item 1 e a recuperação da fila no ambiente local. Os itens de RDS e ElastiCache são ensaio de nuvem.

4. Desempenho

Metas e o caminho que as sustenta:

Meta	Caminho quente	O que não pode entrar nesse caminho
POST p95 < 200 ms	Validar JWT (JWKS em cache), um commit com dois ou três inserts	Publish SNS, chamada ao consolidado, `SUM` de saldo
GET saldo p95 < 100 ms	JWT em cache + GET Redis	`SUM` no hit; no miss o p95 da meta de cache não se aplica e vale o p99 < 300 ms
50 req/s, erro < 1%	2 tasks, ρ ≈ 17%, pool folgado	Rate limit do WAF abaixo do tráfego real; pool de 1 conexão

Premissas de tempo usadas na conta (a confirmar no k6, não tratadas como medida): S_hit 40 ms, S_miss 120 ms, hit ratio 80%, S médio 56 ms, L = 2,8. Se o k6 mostrar S_hit de 80 ms, L dobra e continua pequeno; a decisão de duas tasks não muda. Se mostrar que uma task não sustenta 150 req/s, o mínimo de HA continua 2 e o μ da planilha é que se corrige.

5. Plano de teste de carga (k6)

Script em `tests/load`, alvo `make load`. Ambiente: Docker local com os dois serviços e, quando houver, o dev AWS. Não se aponta k6 para produção.

Cenários:

Cenário	Carga	Critério de sucesso
Leitura nominal	50 req/s de GET diário por 10 min, cache quente, token válido	`http_req_failed` < 1%; p95 < 100 ms
Leitura no teto do enunciado	O mesmo, limiar de falha relaxado só neste cenário	erro < 5%; zero crescimento de DLQ
N−1	Uma task do consolidado, 50 req/s por 5 min	erro < 1%; p99 < 300 ms
Escrita	20 req/s de POST com `Idempotency-Key` única por iteração, por 5 min	p95 < 200 ms; erro < 1%; linhas de outbox não publicadas voltam a zero em < 10 s depois do fim
Escrita com consolidado parado	20 req/s de POST, consumer down	p95 do POST < 200 ms; erro do POST < 1%; fila cresce; ao religar, fila drena
Cache frio	GET com Redis flush no início	erro < 1%; p99 < 300 ms; conexões do Hikari abaixo de 10 por task

O relatório do k6 fica como evidência do RNF-05. A corrida de 1 minuto no Docker local, com os números medidos, está em testes. Ela não substitui os 10 minutos nem o N−1 com duas tasks.

6. Matriz de rastreabilidade

RNF	Mecanismo	Evidência
RNF-01 disponibilidade de lançamentos 99,9%	Service ECS separado, ALB target group próprio, outbox	Alarme de 5xx; `make resiliencia` com consolidado parado e POST 201

RNF	Mecanismo	Evidência
RNF-02 consolidado 99,5%	Multi-AZ, 2 tasks, readiness	Alarme de 5xx do target group do consolidado
RNF-03 latência do POST	Commit local, sem SNS no caminho	k6 cenário de escrita; histograma HTTP
RNF-04 latência do GET	Cache-aside	k6 local de 1 min: p95 6,19 ms e p99 9,48 ms no GET, um container (11)
RNF-05 50 req/s e perda	2 tasks, ρ ≈ 17%, meta interna 1%	Mesma corrida: 50 GET/s, `http_req_failed` 0,00%. A topologia de duas tasks continua por revisar na AWS
RNF-06 lag < 5 s	Relay 500 ms, fila, consumidor concorrente	Métrica de lag; smoke que espera o saldo
RNF-07 durabilidade	Transação única lançamento+outbox, RDS PITR	Teste de integração com falha injetada depois do commit e antes do publish
RNF-08 retenção 14 dias	Atributo da fila	`make tf-validate` e revisão do módulo SQS; teste de atributo no LocalStack
RNF-09 RPO/RTO	PITR 5 min, runbook de restore	Game day de restore em dev, cronometrado
RNF-10 isolamento	Sem cliente HTTP entre os serviços	ArchUnit ou teste que falha se o módulo de lançamentos depender do pacote do consolidado; mais `make resiliencia`
RNF-11 segurança	OIDC, escopos, WAF, KMS	06; testes 401/403/404 cruzado
RNF-12 observabilidade	OTel, alarmes de fila e DLQ	Trace do POST até o consumidor no Grafana local; alarme com ação no Terraform
RNF-13 manutenibilidade	Hexagonal, ArchUnit, `/api/v1`, `eventVersion`	Build com ArchUnit; contrato do evento em teste
RNF-14 custo	Fargate ARM, RDS medium em prod, fila gerenciada, região `us-east-1`, REST API	08: cerca de USD 778/mês em prod, USD 147 em dev e USD 359 em hml, 24×7
RNF-15 LGPD	Transferência internacional com salvaguarda, evento sem descrição	ADR 0021; revisão do payload v1
RNF-16 escala	Auto scaling. Teto 10 nos dois serviços. Backlog da fila por target tracking	Revisão do módulo ECS. Teste de política em dev (subir CPU artificial) ainda não executado

Itens cuja evidência depende de infra que outro fluxo ainda está criando ficam com o teste nomeado. Quando o teste existir, a matriz não muda de mecanismo — muda de “especificado” para “executado”. O estado desta entrega é especificado e amarrado ao comando; a execução contínua é do CI descrito em testes.